CORTEX AI

Private Meeting Intelligence for Actionable Decision-Making

Turn every meeting transcript into structured decisions, tasks,
risks and accountable workflows.

github.com/harshan490/CORTEX-AI

harshan490 | divya-m984 | Nandhini Raju

The Problem

Meetings generate knowledge.
Organizations lose it.

Buried Decisions

Critical decisions are lost in hours of transcript text

Orphaned Action Items

Tasks discussed but never assigned or tracked

Forgotten Deadlines

Dependencies and timelines fade from memory

Manual Follow-Up

Teams spend hours re-reading transcripts to find commitments

Privacy Concerns

Cloud AI means sensitive meeting content leaves your control

No Execution Visibility

No way to track whether meeting outcomes were delivered

Our Solution

Privacy-First

Ollama runs locally — transcripts never leave your machine

Beyond Summarization

Extracts decisions, tasks, risks, dependencies, and clarifications

Human-in-the-Loop

AI extracts, humans approve — nothing is auto-completed

Execution Tracking

Workflow pipeline with progress, analytics, and history

Product Walkthrough

1 Create Meeting

Set title, date, and meeting details

[Screenshot Placeholder]

2 Paste Transcript

Input raw meeting transcript text

[Screenshot Placeholder]

3 Process with AI

Local Ollama extracts structured intelligence

[Screenshot Placeholder]

4 Review Results

Inspect decisions, tasks, risks, dependencies

[Screenshot Placeholder]

5 Track Workflow

Monitor processing stages and approval status

[Screenshot Placeholder]

6 Explore Analytics

View metrics, history, and search knowledge

[Screenshot Placeholder]

What Cortex Extracts

Summary

Concise meeting overview with confidence score

Participants

Speaker names and roles extracted from transcript

Action Items

Tasks with owners, deadlines, priority and evidence

Decisions

What was agreed, by whom, with supporting evidence

Risks

Severity, likelihood, owner, and mitigation notes

Dependencies

What blocks what — from_item to_item with type

Clarifications

Unanswered questions and ambiguities needing follow-up

Confidence Scores

Per-item and overall extraction confidence (0.0 – 1.0)

Example: Project Atlas Launch Meeting

Action Item: "Ravi completes load testing by August 9" | Assignee: Ravi | Confidence: 0.85

Decision: "Launch conditional on security and performance approval" | Decided by: Asha

Risk: "High-priority authentication vulnerability" | Severity: High | Owner: Meera

Dependency: "Production deployment blocked until load test and security audit approved"

Workflow Pipeline

Persistent WorkflowState tracks every processing stage with progress percentages

transcript_received 5%

Workflow created, attempt tracked

transcript_validation 10%

Segments verified non-empty

provider_health_check 20%

Ollama reachability + model check

intelligence_extraction 30%

Single structured LLM call

result_validation 70%

Schema validation of JSON output

database_persistence 80%

Idempotent write to PostgreSQL

awaiting_review 95%

Human approval boundary

completed 100%

Approved by reviewer

Status Values

processing | awaiting_review | completed | failed

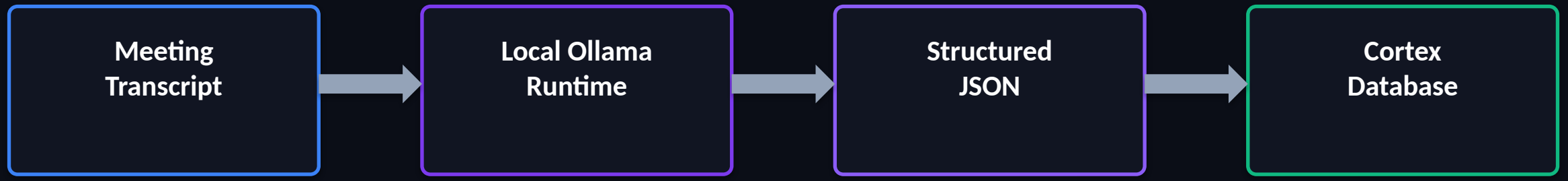
Failure Handling

Error preserved, status set to 'failed', retry increments attempt counter

Idempotent

UniqueConstraint on meeting_id prevents duplicate workflows

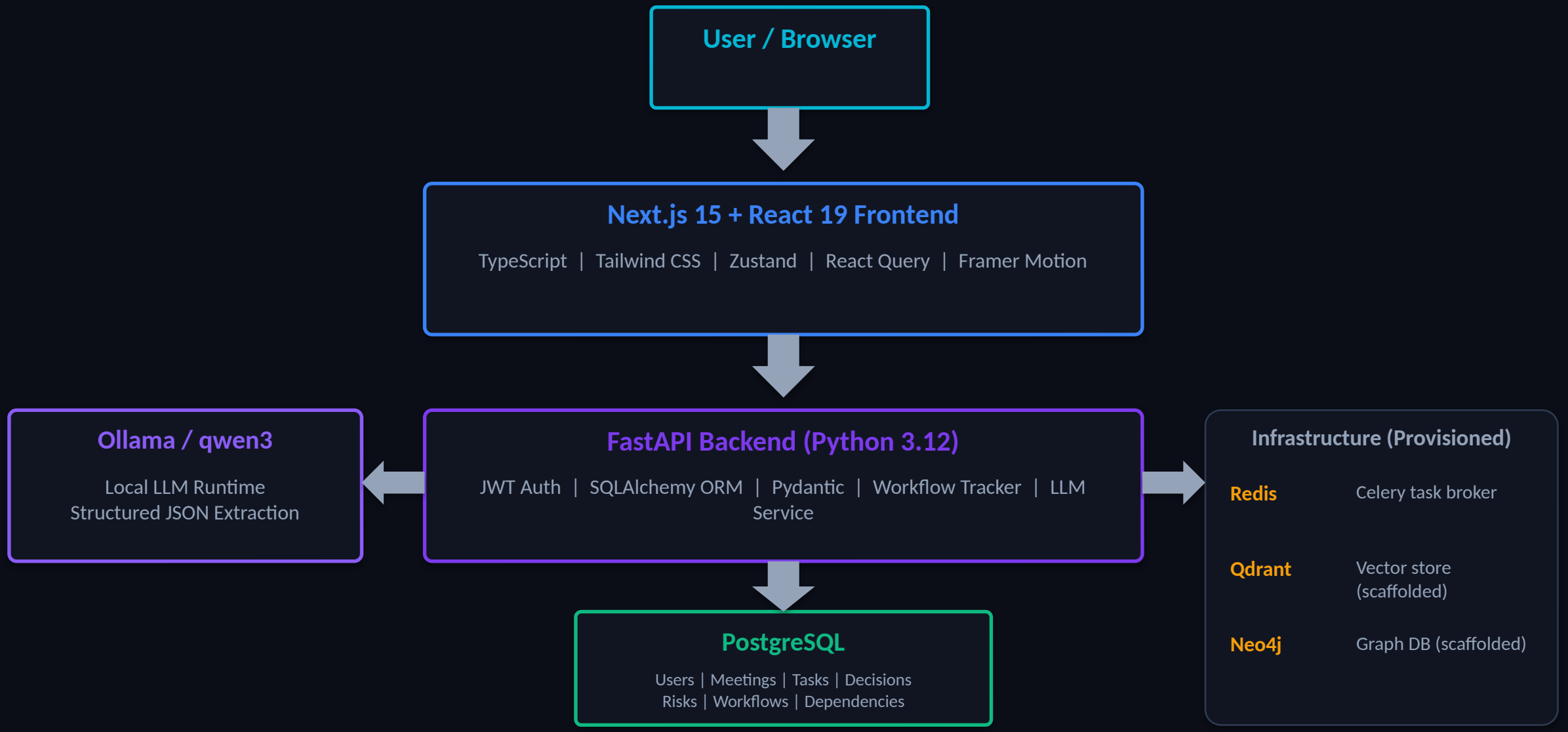
Privacy-First Local AI



Model	qwen3:4b-instruct	**Schema**	Dereferenced $defs/$ref for Ollama grammar performance
Runtime	Ollama (local)	**Timeout**	Configurable (LLM_TIMEOUT_SECONDS) with retry support
Output	Structured JSON via schema	**Providers**	mock | ollama | openai via LLM_PROVIDER setting
Health Check	Model availability verified	**Prompt Injection**	Transcript treated as data only; instructions in transcript ignored

Honest Note: Local CPU inference takes ~1-3 minutes per transcript. Speed depends on hardware. GPU recommended for production.

System Architecture



Data and API Flow

1. **POST /api/auth/login**
   Authenticate, receive JWT token

2. **POST /api/meetings/**
   Create meeting with title and date

3. **POST /api/meetings/{id}/transcript**
   Upload transcript segments as JSON body

4. **POST /api/meetings/{id}/process**
   Trigger AI processing pipeline

5. **WorkflowState created**
   Stages: validation -> health check -> extraction -> persistence

6. **Ollama /api/chat**
   Structured JSON extraction with dereferenced schema

7. **Database persistence**
   Participants, actions, decisions, risks, deps, clarifications

8. **Status: awaiting_review (95%)**
   Frontend polls GET /api/meetings/{id} for updated state

9. **PUT /api/meetings/{id}**
   {"status": "completed"} to approve, "scheduled" to reject

10. **GET /api/analytics/overview**
    User-scoped metrics with period filter (week/month/quarter)

Analytics, History and Search

Analytics Dashboard

- Period filters: Week | Month | Quarter
- User-scoped meeting metrics
- Total meetings, tasks, action items
- Completion rates and decisions count
- Overdue items and critical risks
- Average meeting duration
- Meeting trend data over time
- Team performance scores

Meeting History

- Processed meeting records
- Status tracking per meeting
- Participant lists
- Decision and action item counts
- Risk and dependency counts
- Confidence scores
- Approval/rejection outcomes
- Workflow state visibility

Search

- Full-text search across entities
- Search meetings by title/summary
- Search decisions by title/description
- Search tasks and action items
- Filter by type and date range
- Relevance scoring per result
- Paginated results
- Recent searches tracking

Reliability and Engineering Quality

Authentication

JWT-based auth with bcrypt password hashing and user-scoped data

Error Handling

OllamaError with safe user-facing messages; failures persisted to DB

Test Coverage

71 backend tests + 96 frontend tests across 13 test files

Build Quality

0 TypeScript errors, 0 lint warnings, 0 build errors in production

Mocked LLM Provider

Deterministic mock for automated tests; real Ollama tests opt-in

Database Migrations

Idempotent startup migrations with non-destructive schema evolution

Type Safety

TypeScript strict mode + Pydantic schema validation end-to-end

Transactional Tests

ASGI transport with session rollback isolation — no test pollution

Idempotent Processing

Workflow upsert + data cleanup before re-extraction prevents duplicates

Retry Tracking

WorkflowState.attempt counter; configurable LLM max_retries

Key Technical Challenges

Challenge: **Slow Local CPU Inference**

Solution: Long configurable timeout, workflow progress tracking, duplicate-submission prevention, and planned async execution queue

Challenge: **Reliable Structured LLM Output**

Solution: JSON schema prompting, $defs/$ref dereferencing for Ollama grammar performance, Pydantic validation, and graceful failure handling

Challenge: **Workflow Visibility**

Solution: Persistent WorkflowState model with 14 stage definitions, progress percentages, retry tracking, and human approval boundary

Challenge: **Test Isolation and Flaky LLM Tests**

Solution: ASGI transport, mocked intelligence extraction, transactional rollback, and opt-in real Ollama tests via environment variable

Challenge: **Database Schema Evolution**

Solution: Idempotent, non-destructive startup migrations that add columns and tables without dropping existing data

Innovation and Differentiation

Local-First Intelligence

Transcripts processed by a local model.
No data leaves your infrastructure.

Beyond Summarization

Extracts 8 categories of structured
knowledge, not just a text summary.

Conversation to Execution

Converts discussion into accountable
tasks, owners, and deadlines.

Human Approval Gate

AI extracts, humans approve.
Nothing reaches 'completed' without review.

Persistent Workflow State

Every processing stage tracked with
progress, failures, and retry attempts.

Provider Independence

Swap between mock, Ollama, and OpenAI
with a single environment variable.

Roadmap

Future Work

Near Term

- Background processing queue (Celery)
- Live progress updates (WebSocket / SSE)
- Retry and cancel controls in UI
- Export meeting reports (PDF / CSV)
- Improved speaker diarization

Mid Term

- Google Meet / Zoom / Teams ingestion
- Slack, Jira, and Notion integrations
- Email and calendar action dispatch
- Semantic meeting search (Qdrant)
- Multi-user collaboration features

Long Term

- Organizational knowledge graph (Neo4j)
- Cross-meeting dependency tracking
- Team participation analytics
- Policy and compliance workflows
- Multi-model deployment and routing

From Conversation to Execution

Cortex AI ensures meetings do not end as transcripts —
they become decisions, owners, and measurable execution.

Live Demo Sequence

Login > Create Meeting > Paste Transcript > Process > Review Results > Workflows > Analytics > History > Settings

github.com/harshan490/CORTEX-AI

harshan490 | divya-m984 | Nandhini Raju

Thank You — Questions?